



JAVASCRIPT BASIC

JS Introduction
JS Getting Started
JS Syntax
JS Variables
JS Generating Output
JS Data Types
JS Operators
JS Events
JS Strings
JS Numbers
JS If...Else
JS Switch...Case
JS Arrays
JS Sorting Arrays
JS Loops
JS Functions
JS Objects

JAVASCRIPT & DOM

JS DOM Nodes
JS DOM Selectors
JS DOM Styling
JS DOM Get Set Attributes
JS DOM Manipulation
JS DOM Navigation

JAVASCRIPT & BOM

JS Window
JS Screen
JS Location
JS History
JS Navigator
JS Dialog Boxes
JS Timers

JAVASCRIPT ADVANCED

JS Date and Time
JS Math Operations
JS Type Conversions
JS Event Listeners
JS Event Propagation
JS Borrowing Methods
JS Hoisting Behavior
JS Closures
JS Strict Mode
JS JSON Parsing

JavaScript ES6 Features

< >

In this tutorial you will learn about the new features in latest version of JavaScript.

What is ECMAScript 6 (or ES6)

ECMAScript 2015 (or ES6) is the sixth and major edition of the ECMAScript language specification standard. It defines the standard for the JavaScript implementation.

ES6 brought significant changes to the JavaScript language. It introduces several new features such as, block-scoped variables, new loop for iterating over arrays and objects, template literals, and many other enhancements to make JavaScript programming easier and more fun. In this chapter, we will discuss some of the best ES6 features that you can use in your everyday JavaScript coding.

The let Keyword

ES6 introduces the new `let` keyword for declaring variables. Prior to ES6, the only way to declare a variable in JavaScript was the `var` keyword. Let's see what's the difference between them is.

There are two critical differences between the `var` and `let`. Variables declared with the `var` keyword are **function-scoped** and **hoisted** at the top within its scope, whereas variables declared with `let` keyword are block-scoped (`{ }`) and they are not hoisted.

Block scoping simply means that a new scope is created between a pair of curly brackets i.e. `{ }`. Therefore, if you declare a variable with the `let` keyword inside a loop, it does not exist outside of the loop, as demonstrated in the following example:

Example**Try this code »**

```
1 // ES6 syntax
2 for(let i = 0; i < 5; i++) {
3     console.log(i); // 0,1,2,3,4
4 }
5 console.log(i); // undefined
6
7
8 // ES5 syntax
9 for(var i = 0; i < 5; i++) {
10    console.log(i); // 0,1,2,3,4
11 }
12 console.log(i); // 5
```

As you can see in the above example the variable `i` in the first block is not accessible outside the `for` loop. This also enables us to reuse the same variable name multiple times as its scope is limited to the block (`{ }`), which results in less variable declaration and more cleaner code.

The const Keyword

[JS Error Handling](#)
[JS Regular Expressions](#)
[JS Form Validation](#)
[JS Cookies](#)
[JS AJAX Requests](#)
[JS ES6 Features](#)

JAVASCRIPT EXAMPLES

[JS Practice Examples](#)
[JS FAQ's Answers](#)

JAVASCRIPT REFERENCE

[JS Properties and Methods](#)
[JS Reserved Keywords](#)
[More references](#)

The new `const` keyword makes it possible to define constants. Constants are read-only, you cannot reassign new values to them. They are also block-scoped like `let`.

Example	Try this code »
<pre> 1 const PI = 3.14; 2 console.log(PI); // 3.14 3 4 PI = 10; // error </pre>	

However, you can still change object properties or array elements:

Example	Try this code »
<pre> 1 // Changing object property value 2 const PERSON = {name: "Peter", age: 28}; 3 console.log(PERSON.age); // 28 4 PERSON.age = 30; 5 console.log(PERSON.age); // 30 6 7 // Changing array element 8 const COLORS = ["red", "green", "blue"]; 9 console.log(COLORS[0]); // red 10 COLORS[0] = "yellow"; 11 console.log(COLORS[0]); // yellow </pre>	

The for...of Loop

The new `for...of` loop allows us to iterate over arrays or other iterable objects very easily. Also, the code inside the loop is executed for each element of the iterable object. Here's an example:

Example	Try this code »
<pre> 1 // Iterating over array 2 let letters = ["a", "b", "c", "d", "e", "f"]; 3 4 for(let letter of letters) { 5 console.log(letter); // a,b,c,d,e,f 6 } 7 8 // Iterating over string 9 let greet = "Hello World!"; 10 11 for(let character of greet) { 12 console.log(character); // H,e,l,l,o, ,W,o,r,l,d,! 13 } </pre>	

The `for...of` loop doesn't work with objects because they are not iterable. If you want to iterate over the properties of an object you can use the `for-in` loop.

Template Literals

Template literals provide an easy and clean way create multi-line strings and perform string interpolation. Now we can embed variables or expressions into a string at any spot without any hassle.

Template literals are created using back-tick (```) (grave accent) character instead of the usual double or single quotes. Variables or expressions can be placed inside the string using the `${...}` syntax. Compare the following examples and see how much useful it is:

Example	Try this code »
<pre>1 // Simple multi-line string 2 let str = `The quick brown fox 3   jumps over the lazy dog.`; 4 5 // String with embedded variables and expression 6 let a = 10; 7 let b = 20; 8 let result = `The sum of \${a} and \${b} is \${a+b}.`; 9 console.log(result); // The sum of 10 and 20 is 30.</pre>	

While in ES5, to achieve the same we had to write something like this:

Example	Try this code »
<pre>1 // Multi-line string 2 var str = 'The quick brown fox\n\t' 3 + 'jumps over the lazy dog.'; 4 5 // Creating string using variables and expression 6 var a = 10; 7 var b = 20; 8 var result = 'The sum of ' + a + ' and ' + b + ' is ' + (a+b) + '.'; 9 console.log(result); // The sum of 10 and 20 is 30.</pre>	

Default Values for Function Parameters

Now, in ES6 you can specify default values to the [function parameters](#). This means that if no arguments are provided to function when it is called these default parameters values will be used. This is one of the most awaited features in JavaScript. Here's an example:

Example	Try this code »
<pre>1 function sayHello(name='World') { 2 return `Hello \${name}!`; 3 } 4 5 console.log(sayHello()); // Hello World! 6 console.log(sayHello('John')); // Hello John!</pre>	

While in ES5, to achieve the same we had to write something like this:

Example	Try this code »
<pre>1 function sayHello(name) { 2 var name = name 'World'; 3 return 'Hello ' + name + '!'; 4 } 5 6 console.log(sayHello()); // Hello World! 7 console.log(sayHello('John')); // Hello John!</pre>	

Arrow Functions

Arrow Functions are another interesting feature in ES6. It provides a more concise syntax for writing **function expressions** by opting out the `function` and `return` keywords.

Arrow functions are defined using a new syntax, the fat arrow (`=>`) notation. Let's see how it looks:

Example	Try this code »
<pre>1 // Function Expression 2 var sum = function(a, b) { 3 return a + b; 4 } 5 console.log(sum(2, 3)); // 5 6 7 // Arrow function 8 var sum = (a, b) => a + b; 9 console.log(sum(2, 3)); // 5</pre>	

As you can see there is no `function` and `return` keyword in arrow function declaration.

You can also skip the parentheses i.e. `()` in case when there is exactly one parameter, but you will always need to use it when you have zero or more than one parameter.

Additionally, if there's more than one expression in the function body, you need to wrap it braces `{ }`. In this case you also need to use the `return` statement to return a value.

There are several variations of how you can write arrow functions. Here are the most commonly used:

Example	Try this code »
<pre>1 // Single parameter, single statement 2 var greet = name => alert("Hi " + name + "!"); 3 greet("Peter"); // Hi Peter! 4 5 // Multiple arguments, single statement 6 var multiply = (x, y) => x * y; 7 alert(multiply(2, 3)); // 6 8 9 10 // Single parameter, multiple statements 11 var test = age => { 12 if(age > 18) { 13 alert("Adult"); 14 } else { 15 alert("Teenager"); 16 } 17 } 18 test(21); // Adult 19 20 // Multiple parameters, multiple statements 21 var divide = (x, y) => { 22 if(y !== 0) { 23 return x / y; 24 } 25 } 26 alert(divide(10, 2)); // 5 27 28 // No parameter, single statement 29 var hello = () => alert('Hello World!'); 30 hello(); // Hello World!</pre>	

There is an important difference between regular functions and arrow functions. Unlike a normal function, an arrow function does not have its own `this`, it takes `this` from the outer function where it is defined. In JavaScript, `this` is the current execution context of a function.

To understand this clearly, let's check out the following examples:

Example[Try this code »](#)

```
1 function Person(nickname, country) {
2   this.nickname = nickname;
3   this.country = country;
4
5   this.getInfo = function() {
6     // Outer function context (Person object)
7     return function() {
8       // Inner function context (Global object 'Window')
9       alert(this.constructor.name); // Window
10      alert("Hi, I'm " + this.nickname + " from " + this.country);
11    };
12  }
13 }
14
15 var p = new Person('Rick', 'Argentina');
16 var printInfo = p.getInfo();
17 printInfo(); // Hi, I'm undefined from undefined
```

Rewriting the same example using ES6 template literals and arrow function:

Example[Try this code »](#)

```
1 function Person(nickname, country) {
2   this.nickname = nickname;
3   this.country = country;
4
5   this.getInfo = function() {
6     // Outer function context (Person object)
7     return () => {
8       // Inner function context (Person object)
9       alert(this.constructor.name); // Person
10      alert(`Hi, I'm ${this.nickname} from ${this.country}`);
11    };
12  }
13 }
14
15 let p = new Person('Rick', 'Argentina');
16 let printInfo = p.getInfo();
17 printInfo(); // Hi, I'm Rick from Argentina
```

As you can clearly see, the `this` keyword in the above example refers to the context of the function enclosing the arrow function which is Person object (*line no-9*), unlike the previous example where it referred to the global object `Window` (*line no-9*).

Classes

In ECMAScript 5 and earlier, classes were never existed in JavaScript. Classes are introduced in ES6 which looks similar to classes in other object oriented languages, such as Java, PHP, etc., however they do not

work exactly the same way. ES6 classes make it easier to create objects, implement inheritance by using the `extends` keyword, and reuse the code.

In ES6 you can declare a class using the new `class` keyword followed by a class-name. By convention class names are written in TitleCase (i.e. capitalizing the first letter of each word).

Example[Try this code »](#)

```
1  class Rectangle {
2    // Class constructor
3    constructor(length, width) {
4      this.length = length;
5      this.width = width;
6    }
7
8    // Class method
9    getArea() {
10     return this.length * this.width;
11   }
12 }
13
14 // Square class inherits from the Rectangle class
15 class Square extends Rectangle {
16   // Child class constructor
17   constructor(length) {
18     // Call parent's constructor
19     super(length, length);
20   }
21
22   // Child class method
23   getPerimeter() {
24     return 2 * (this.length + this.width);
25   }
26 }
27
28 let rectangle = new Rectangle(5, 10);
29 alert(rectangle.getArea()); // 50
30
31 let square = new Square(5);
32 alert(square.getArea()); // 25
33 alert(square.getPerimeter()); // 20
34
35 alert(square instanceof Square); // true
36 alert(square instanceof Rectangle); // true
37 alert(rectangle instanceof Square); // false
```

In the above example the Square class inherits from Rectangle using the `extends` keyword. Classes that inherit from other classes are referred to as derived classes or child classes.

Also, you must call `super()` in the child class constructor before accessing the context (`this`). For instance, if you omit the `super()` and call the `getArea()` method on square object it will result in an error, since `getArea()` method require access to `this` keyword.



Note: Unlike function declarations, class declarations are not hoisted. Class declarations reside in the temporal dead zone (TDZ) until the execution reaches the point of class declaration, similar to `let` and `const` declarations. Therefore, you need to declare your class before accessing it, otherwise a `ReferenceError` will occur.

Modules

Prior to ES6, there were no native support for modules in JavaScript. Everything inside a JavaScript application, for example variables across different JavaScript files, shared the same scope.

ES6 introduces file based module, in which each module is represented by a separate `.js` file. Now, you can use the `export` or `import` statement in a module to export or import variables, functions, classes or any other entity to/from other modules or files.

Let's create a module i.e. a JavaScript file "main.js" and place the following code in it:

Example	Try this code »
<pre>1 let greet = "Hello World!"; 2 const PI = 3.14; 3 4 function multiplyNumbers(a, b) { 5 return a * b; 6 } 7 8 // Exporting variables and functions 9 export { greet, PI, multiplyNumbers };</pre>	

Now create another JavaScript file "app.js" with the following code:

Example	Try this code »
<pre>1 import { greet, PI, multiplyNumbers } from './main.js'; 2 3 alert(greet); // Hello World! 4 alert(PI); // 3.14 5 alert(multiplyNumbers(6, 15)); // 90</pre>	

Finally create a HTML file "test.html" and with the following code and open this HTML file in your browser using HTTP protocol (or use localhost). Also notice the `type="module"` on script tag.

Example	Try this code »
<pre>1 <!DOCTYPE html> 2 <html lang="en"> 3 <head> 4 <meta charset="utf-8"> 5 <title>ES6 Module Demo</title> 6 </head> 7 <body> 8 <script type="module" src="app.js"></script> 9 </body> 10 </html></pre>	

The Rest Parameters

ES6 introduces rest parameters that allow us to pass an arbitrary number of parameters to a function in the form of an array. This is particularly helpful in situations when you want to pass parameters to a function but you have no idea how many you will need.

A rest parameter is specified by prefixing a named parameter with rest operator (`...`) i.e. three dots. Rest parameter can only be the last one in the list of parameters, and there can only be one rest parameter. Take a look at the following example, to see how it works:

Example	Try this code »
<pre> 1 function sortNames(...names) { 2 return names.sort(); 3 } 4 5 alert(sortNames("Sarah", "Harry", "Peter")); // Harry, Peter, Sarah 6 alert(sortNames("Tony", "Ben", "Rick", "Jos")); // John, Jos, Rick, Tony </pre>	

When the rest parameter is the only parameter in a function, it gets all the arguments passed to the function, otherwise it gets the rest of the arguments that exceeds the number of named parameters.

Example	Try this code »
<pre> 1 function myFunction(a, b, ...args) { 2 return args; 3 } 4 5 alert(myFunction(1, 2, 3, 4, 5)); // 3,4,5 6 alert(myFunction(-7, 5, 0, -2, 4.5, 1, 3)); // 0,-2,4.5,1,3 </pre>	



Note: Don't confuse the term rest parameters with the REST (REpresentational State Transfer).

☞ This has nothing to do with RESTful web services.

The Spread Operator

The spread operator, which is also denoted by (`...`), performs the exact opposite function of the rest operator. The spread operator spreads out (i.e. splits up) an array and passes the values into the specified function, as shown in the following example:

Example	Try this code »
<pre> 1 function addNumbers(a, b, c) { 2 return a + b + c; 3 } 4 5 let numbers = [5, 12, 8]; 6 7 // ES5 way of passing array as an argument of a function 8 alert(addNumbers.apply(null, numbers)); // 25 9 10 // ES6 spread operator 11 alert(addNumbers(...numbers)); // 25 </pre>	

The spread operator can also be used to insert the elements of an array into another array without using the array methods like `push()`, `unshift()`, `concat()`, etc.

Example	Try this code »
<pre> 1 let pets = ["Cat", "Dog", "Parrot"]; 2 let bugs = ["Ant", "Bee"]; 3 </pre>	


```

4 // Creating an array by inserting elements from other arrays
5 let animals = [...pets, "Tiger", "Wolf", "Zebra", ...bugs];
6
7 alert(animals); // Cat,Dog,Parrot,Tiger,Wolf,Zebra,Ant,Bee

```

Destructuring Assignment

The destructuring assignment is an expression that makes it easy to extract values from arrays, or properties from objects, into distinct variables by providing a shorter syntax.

There are two kinds of destructuring assignment expressions—the *array* and *object* destructuring assignment. Well, let's see how each of them exactly works:

The array destructuring assignment

Prior to ES6, to get an individual value of an array we need to write something like this:

Example	Try this code »
<pre> 1 // ES5 syntax 2 var fruits = ["Apple", "Banana"]; 3 4 var a = fruits[0]; 5 var b = fruits[1]; 6 alert(a); // Apple 7 alert(b); // Banana </pre>	

In ES6, we can do the same thing in just one line using the array destructuring assignment:

Example	Try this code »
<pre> 1 // ES6 syntax 2 let fruits = ["Apple", "Banana"]; 3 4 let [a, b] = fruits; // Array destructuring assignment 5 6 alert(a); // Apple 7 alert(b); // Banana </pre>	

You can also use [rest operator](#) in the array destructuring assignment, as shown here:

Example	Try this code »
<pre> 1 // ES6 syntax 2 let fruits = ["Apple", "Banana", "Mango"]; 3 4 let [a, ...r] = fruits; 5 6 alert(a); // Apple 7 alert(r); // Banana,Mango 8 alert(Array.isArray(r)); // true </pre>	

The object destructuring assignment

In ES5 to extract the property values of an object we need to write something like this:

Example	Try this code »

```

1 // ES5 syntax
2 var person = {name: "Peter", age: 28};
3
4 var name = person.name;
5 var age = person.age;
6
7 alert(name); // Peter
8 alert(age); // 28

```

But in ES6, you can extract object's property values and assign them to the variables easily like this:

Example

[Try this code »](#)

```

1 // ES6 syntax
2 let person = {name: "Peter", age: 28};
3
4 let {name, age} = person; // Object destructuring assignment
5
6 alert(name); // Peter
7 alert(age); // 28

```

Most of the features we've discussed above are supported in the latest version of the major web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, etc.

Alternatively, you can use the online transpilers (source-to-source compilers) like [Babel](#) free of cost to transpile your current ES6 code to ES5 for better browser compatibility without leaving out the benefits of enhanced syntax and capabilities of ES6.

[« PREVIOUS PAGE](#)
[NEXT PAGE »](#)

Bootstrap Code Snippets

Free ready to use UI design elements, templates and code snippets

[Browse Snippets](#)


Is this website helpful to you? Please give us a like, or share your feedback to help us improve. Connect with us on Facebook and Twitter for the latest updates.

ABOUT US

Our Story
Terms of Use
Privacy Policy

CONTACT

Contact Us
Report Error
Advertise

INTERACTIVE TOOLS

Bootstrap Icon Search Utility
Title & Meta Length Calculator
Bootstrap Button Generator
Font Awesome Icon Finder
HTML Formatter
HTML Color Picker
SQL Playground
HTML Editor

